AgriFlow Farming: Data-Empowered Crops & Irrigation

Docker Locker Final Project

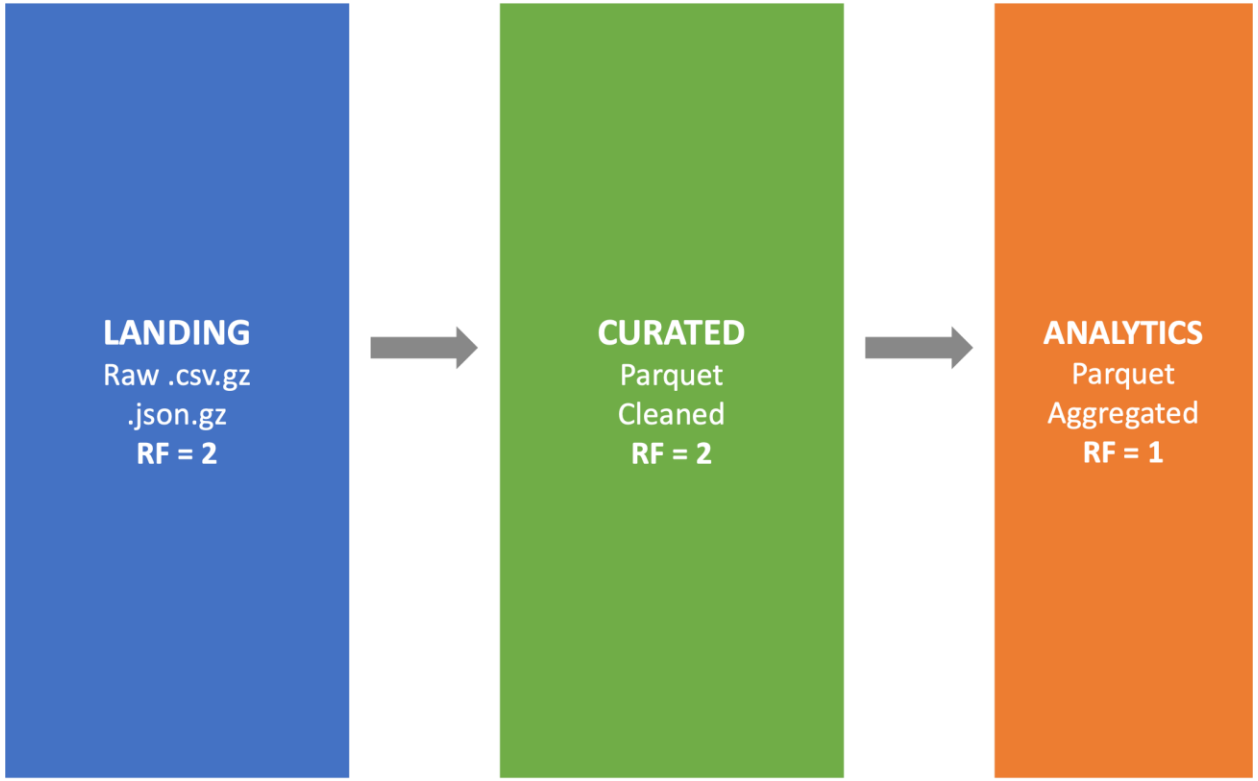
Freeman Talla, Gustavo Storck, Hesham Rashid,
Savannah Day, Zachary Zambrana

Business Problem

- AgriFlow Farming operates thousands of acres generating massive volumes of data from soil sensors, weather stations, and irrigation controllers
- Field managers historically relied on intuition rather than a unified data infrastructure
- No centralized pipeline meant the company could not systematically analyze or act on its own data
- Two core business questions drove the project:
- What combination of soil, weather, and irrigation produces the highest crop yield per acre?
- Where is water being wasted, and how can consumption be reduced without hurting crop health?
- Without answers to these questions, AgriFlow faced continued yield inconsistency and unquantified water expenditure

Part 1 – Data Lake Foundation

- Designed a 3-zone HDFS data lake for AgriFlow
- Ingested all 5 raw datasets via WebHDFS API
- Set replication factors by data criticality
- Round-trip verified: Spark reads directly from HDFS



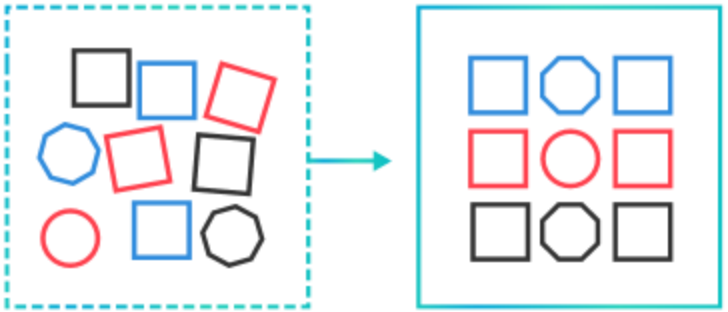
Part 1 – AgriFlow Data Sources

Dataset	Format	Row Count	HDFS Landing Zone Path
Crop-yields	CSV.gz	100,000	/agriflow/landing/crop-records
Equipment-usage	JSON.gz	80,002	/agriflow/landing/equipment-usage
Market-prices	CSV.gz	20,000	/agriflow/landing/market-prices
Soil-sensors	JSON.gz	500,002	/agriflow/landing/soil-sensors
Weather-daily	CSV.gz	150,000	/agriflow/landing/weather-data
Total: 850,004 records successfully ingested into HDFS			

- **Key finding:** soil-sensors (500K rows, 16 MB compressed) contained a corrupt first record detected automatically by Spark — a real data quality issue that shaped our Stage 2 cleaning strategy.

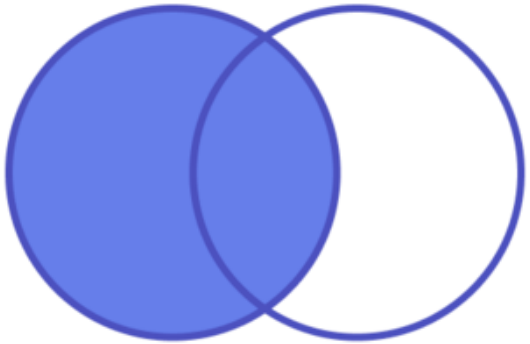
Part 2 – Batch Transformation Pipeline

- Began by reading the raw data from part 1 using PySpark.
- Standardized and aligned mismatched column names
- Formatted dates and numeric in correct formats
- Removed duplicate records and only kept logic records for joins.
- Build daily summaries to view data before joining.
- Data written into parquet files partitioned by year and month for speed



Part 2 - Batch Transformation Pipeline Contd.

- Crop yield, soil, equipment, and weather data were all joined into one enriched table.
- Soil and equipment joined using farm_id, field_id and year because they are field specific.
- Weather joined using farm_id and year because it applies to the whole farm.
- Left joins were used because crop data is the main table which preserves historical data regardless of issues with inconsistent data points.
- In joining, there was a disparity between the historical data and sensor data.



LEFT JOIN

Part 2 - Analytics Output

- Revenue output shows corn to have the strongest upside. Top revenue rows dominated by corn in October across many years and regions. Ex. Midwest south October 2019 $4.13 million
- All crop performances tend to be fairly stable over time. Corn averages ~160 bushels per acre, soybeans around 40, what around 55 which gives a good baseline to detect abnormal years.
- This changes with the limited number of sensor fields, the top of which reached 213 bushels per acre. This shows sensor equipped fields are associated with stronger yields and should be pushed to more fields.
- The equipment efficiency output allows us to monitor operating costs to decide which machines should be used more or less often. Tractors usually have the most hours used but harvesters have the highest fuel per hour at ~ 10.
- These metrics can help decide which tools are necessary and which can be cut back to save money and fuel.

Part 2 - Analytics Output

crop	region	year	month	crop_records	total_yield	avg_price_per_unit	avg_futures_price	market_obs	total_estimated_revenue
corn	midwest_south	2019	10	4403	736043.8000000003	5.747272727272727	6.0609090909090915	11	4230244.457818183
corn	midwest_south	2024	10	4280	711259.5	5.825	5.922999999999999	10	4143086.5875
corn	plains	2023	10	4374	728240.7000000008	5.633846153846155	5.772307692307693	13	4102796.066769236
corn	midwest_central	2020	10	4280	710819.7000000002	5.721818181818183	5.839090909090908	11	4067181.083454547
corn	plains	2021	10	4311	718806.7999999991	5.616470588235295	5.8505882352941185	17	4037157.250823525
corn	midwest_south	2021	10	4311	718806.7999999991	5.610833333333333	5.786666666666668	12	4033105.1536666616
corn	midwest_north	2021	10	4311	718806.7999999991	5.523000000000001	5.648	10	3969969.9563999954
corn	midwest_central	2023	10	4374	728240.7000000008	5.395624999999999	5.55625	16	3929313.7269375036
corn	midwest_central	2017	10	4276	715493.8000000007	5.472857142857143	5.624285714285714	7	3915795.3540000045
corn	midwest_central	2022	10	4402	731106.9999999991	5.352727272727273	5.485454545454545	11	3913416.3781818133
corn	midwest_south	2023	10	4374	728240.7000000008	5.314000000000001	5.566666666666666	15	3869871.0798000046
corn	plains	2019	10	4403	736043.8000000003	5.243076923076924	5.18846153846154	13	3859134.262153848
corn	plains	2018	10	4380	728597.5000000002	5.249999999999999	5.48157894736842	19	3825136.8750000005
corn	plains	2024	10	4280	711259.5	5.3773333333333335	5.492666666666666	15	3824679.418
corn	midwest_north	2020	10	4280	710819.7000000002	5.359230769230769	5.5238461538461525	13	3809446.8076153854
corn	midwest_north	2022	10	4402	731106.9999999991	5.191333333333334	5.310666666666665	15	3795420.1393333287
corn	plains	2022	10	4402	731106.9999999991	5.1899999999999995	5.3353846153846165	13	3794445.329999995
corn	plains	2020	10	4280	710819.7000000002	5.320714285714286	5.362857142857143	14	3782068.5323571437
corn	midwest_north	2017	10	4276	715493.8000000007	5.161666666666666	5.4433333333333325	12	3693140.49766667
corn	midwest_south	2018	10	4380	728597.5000000002	5.065333333333332	5.113333333333333	15	3690589.2033333336

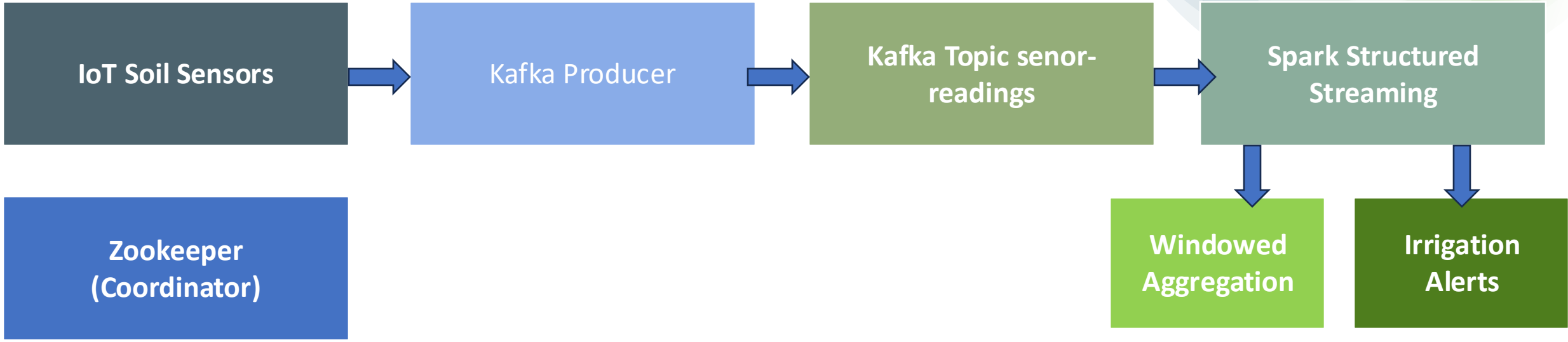
equipment_efficiency

equipment_type	farm_id	field_id	year	month	daily_rows	hours_used	fuel_used	event_count	fuel_per_hour
tractor	FARM-31	FARM-31-F06	2024	6	9	82.8	776.0	12	9.371980676328503
harvester	FARM-41	FARM-41-F19	2024	3	3	22.1	276.6	3	12.515837104072398
tractor	FARM-44	FARM-44-F09	2024	3	5	20.0	168.59999999999997	5	8.429999999999998
sprayer	FARM-22	FARM-22-F18	2024	3	4	9.6	95.3	4	9.927083333333334
harvester	FARM-13	FARM-13-F09	2024	8	3	29.4	240.4	3	8.17687074829932
irrigator	FARM-36	FARM-36-F02	2024	9	2	17.4	174.1	2	10.005747126436782
harvester	FARM-01	FARM-01-F11	2024	5	5	27.7	216.2	5	7.805054151624549
sprayer	FARM-25	FARM-25-F07	2024	7	2	7.3	45.1	2	6.178082191780822
irrigator	FARM-38	FARM-38-F08	2024	4	3	8.5	48.5	3	5.705882352941177
irrigator	FARM-27	FARM-27-F07	2024	4	5	36.5	334.5	6	9.164383561643836
tractor	FARM-47	FARM-47-F04	2024	6	5	24.0	166.2	5	6.925
harvester	FARM-21	FARM-21-F01	2024	5	7	46.0	457.5	7	9.945652173913043
tractor	FARM-47	FARM-47-F05	2024	9	7	52.0	465.2	7	8.946153846153846
tractor	FARM-48	FARM-48-F04	2024	7	5	40.8	366.2	6	8.975490196078432
irrigator	FARM-15	FARM-15-F06	2024	9	7	43.39999999999999	298.4	8	6.87557603686636
irrigator	FARM-37	FARM-37-F07	2024	7	5	24.8	226.6	5	9.137096774193548
harvester	FARM-50	FARM-50-F05	2024	6	4	32.4	329.8	4	10.179012345679013
tractor	FARM-22	FARM-22-F09	2024	6	3	32.900000000000006	323.5	4	9.832826747720363
irrigator	FARM-14	FARM-14-F05	2024	8	7	29.2	284.3	8	9.736301369863014
tractor	FARM-44	FARM-44-F09	2024	5	10	82.39999999999999	833.7	10	10.117718446601943

Part 3 – Kafka + Structured Streaming Pipeline

- Added Kafka and Zookeeper to the Docker stack for real-time messaging.
- Designed two topics: sensor-readings (3 partitions) and irrigation-alerts (1 partition).
- Built a Kafka producer that streams live soil sensor readings into the pipeline.
- Implemented Spark Structured Streaming to consume and process the live data.
- Windowed aggregations compute average moisture, temperature, and pH per farm every 5 minutes.
- Alert logic flags any field with soil moisture below 20% as a LOW_MOISTURE_ALERT in real time.

Part 3 - Batch Transformation Pipeline Contd.



Topic	Event Type	Partitions	Rationale
`sensor-readings`	Soil/temp/humidity IoT readings per field	3	Partitioned by farm_id to parallelize processing across farm zones
`irrigation-alerts`	High water usage or drought-risk triggers	1	Low volume, single alert consumer is sufficient

Part 3 – Streaming Output

=== Windowed Aggregation Results ===

window	farm_id	avg_moisture	avg_temp	avg_ph	sensor_count
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-01	24.6	55.3	6.17	1
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-01	22.0	60.5	6.555	2
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-02	44.1	66.8	6.776666666666666	3
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-02	48.4	62.0	7.255	2
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-03	30.36666666666667	65.56666666666666	7.023333333333333	3
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-04	20.2	62.85	6.800000000000001	2
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-04	19.2	65.9	6.11	1
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-05	30.65	47.349999999999994	6.99	2
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-06	30.8	59.3	6.28	1
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-08	39.900000000000006	60.6	6.166666666666667	3
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-08	38.650000000000006	59.2	6.015	2
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-08	26.6	63.0	5.64	1
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-10	29.549999999999997	63.8	6.5825000000000005	4
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-10	26.23333333333333	68.96666666666667	6.926666666666667	3
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-10	26.4	62.1	7.67	1
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-11	49.2	62.0	6.33	1
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-13	37.1	65.5	7.46	1
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-14	37.45	63.6	7.23	2
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-14	42.1	62.0	6.98	1
{2026-04-26 16:48:00, 2026-04-26 16:53:00}	FARM-15	44.25	69.9	7.035	2

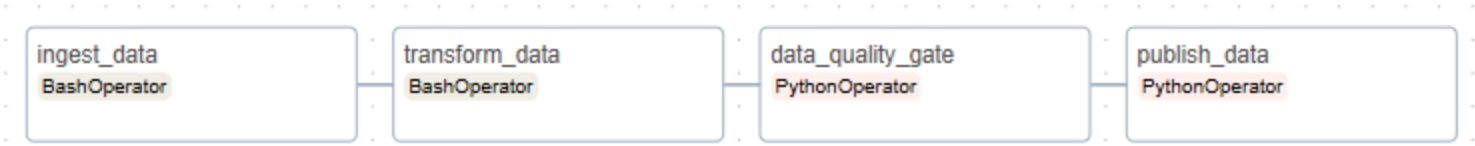
=== Low Moisture Alert Results ===

farm_id	field_id	sensor_id	soil_moisture	event_time	alert_type
FARM-04	FARM-04-F11	SOIL-02129	19.2	2026-04-26 16:52:16.542588	LOW_MOISTURE_ALERT
FARM-20	FARM-20-F17	SOIL-00259	18.1	2026-04-26 16:52:18.83529	LOW_MOISTURE_ALERT
FARM-41	FARM-41-F09	SOIL-01941	13.9	2026-04-26 16:52:29.022213	LOW_MOISTURE_ALERT
FARM-42	FARM-42-F04	SOIL-01956	19.7	2026-04-26 16:52:29.225234	LOW_MOISTURE_ALERT
FARM-39	FARM-39-F08	SOIL-01909	15.1	2026-04-26 16:52:29.631287	LOW_MOISTURE_ALERT
FARM-10	FARM-10-F07	SOIL-03584	14.0	2026-04-26 16:52:20.465049	LOW_MOISTURE_ALERT
FARM-16	FARM-16-F08	SOIL-00903	19.5	2026-04-26 16:52:23.1377	LOW_MOISTURE_ALERT
FARM-16	FARM-16-F04	SOIL-01589	12.3	2026-04-26 16:52:25.448502	LOW_MOISTURE_ALERT
FARM-24	FARM-24-F12	SOIL-03769	18.0	2026-04-26 16:52:30.239991	LOW_MOISTURE_ALERT
FARM-01	FARM-01-F01	SOIL-00001	19.4	2026-04-26 16:52:30.647614	LOW_MOISTURE_ALERT

Part 4 – Airflow Orchestration

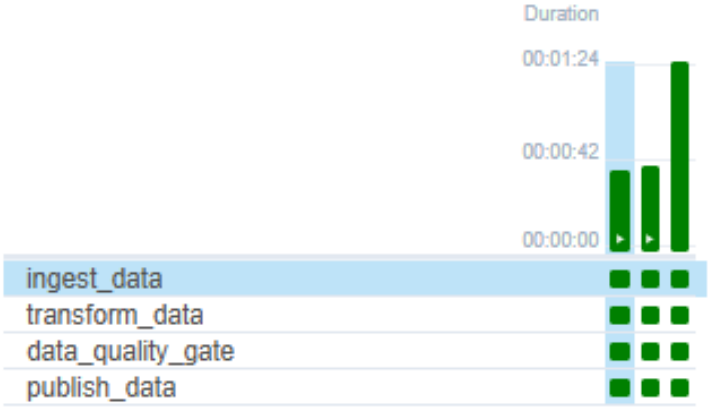
Objective: Orchestrate daily agricultural data processing using Airflow

Pipeline Flow: Ingest → Transform → Quality Gate → Publish



Technologies:
- Apache Airflow (orchestration)
- Apache Spark (processing)
- HDFS (storage)

Key Features:
- Scheduled daily DAG (@daily)
- Idempotent processing (overwrite writes)
- Modular task-based architecture



Part 4 – Quality Checks

We implemented domain-specific quality checks:

1. **Range Validation**
   - Yield values constrained to realistic bounds (0–500 bushels/acre)
2. **Completeness Check**
   - No missing (NULL) values in critical fields
3. **Data Presence / Outage Detection**
   - Ensures farm-level data exists in each batch

```python
# -------------------------
# 3. SENSOR OUTAGE PROXY (missing records)
# -------------------------
farms = df.select("farm_id").distinct().count()

if farms < 1:
    raise Exception("❌ No farm data detected (possible outage)")

print(f"✅ Quality checks passed: {count} rows, {farms} farms")
```

Why it matters:
- Prevents bad data from entering analytics layer
- Ensures reliability of downstream insights

Task Id: data_quality_gate
Status: success
Started: 2026-05-05, 20:25:21 UTC
Duration: 00:00:12
Trigger Rule: all_success

Part 4 – Reliability & Production Readiness

Reliability Features:

Retry Mechanism

Automatic retries for transient failures

SLA Monitoring

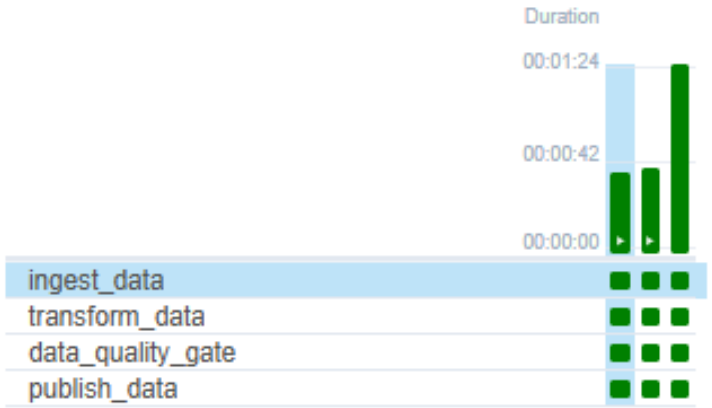
Ensures pipeline completes within expected timeframe

Idempotent Writes

Overwrite mode ensures consistent outputs per run

Task Isolation

Each stage runs independently (Airflow operators)



Outcome:
- Stable, repeatable daily pipeline
- Fault-tolerant batch processing

Business Insights

Sensor-equipped fields reached up to 213 bushels/acre compared to the 160 bushels/acre baseline — a 33% yield uplift

Sandy-Loam soils outperformed Clay-Heavy soils by 15–22% across all crop types and regions

Fields receiving 3–4 drip-irrigation events per week during germination consistently hit the upper range of their yield potential

Corn showed the strongest revenue upside, with October Midwest harvests dominating top revenue outputs — including $4.13 million in a single region in October 2019

12–15% of total irrigation water was identified as waste, broken down into post-rain irrigation (8%), high-moisture over-irrigation (4–5%), and schedule drift (2–3%)

Harvesters consume the most fuel per hour at ~10 gal/hr, flagging an opportunity for fleet optimization

Conclusions

- A well-structured Medallion Architecture ensures data quality at every stage, making downstream analytics reliable and trustworthy
- Real-time streaming and batch processing are complementary — Kafka catches what batch jobs miss between daily runs
- Standardizing environments early with Docker saves significant debugging time across a team
- Abstracting configuration values away from pipeline logic makes the system far more maintainable and portable
- Precision data infrastructure directly translates to business value — sensor-equipped fields proved it with a 33% yield advantage
- If starting over, the team would prioritize cross-machine compatibility and dataset source stability from day one to avoid mid-project disruptions